

fire_demo3_api

Backend API & how to build a frontend on it

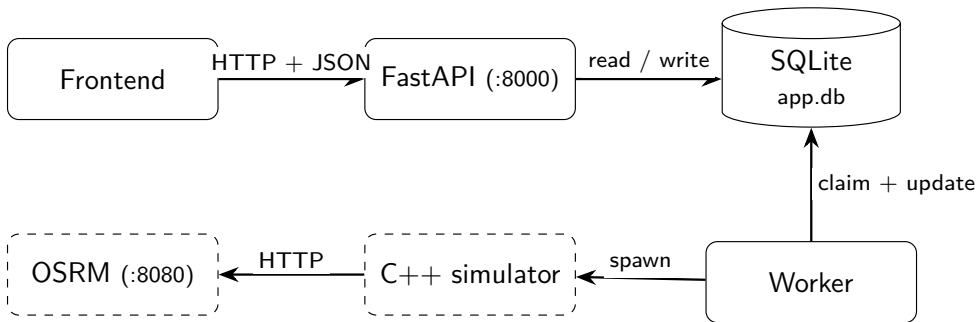
Repo branch: backend-v2

https://github.com/jptalusan/fire_demo3_api/tree/backend-v2

What it is

- ▶ FastAPI backend that drives the C++ fire/EMS simulator.
- ▶ Login with username + password; server returns a session cookie.
- ▶ Long-running simulations go on a queue; a worker process runs them.
- ▶ Frontend reads and writes JSON over HTTP.

How the pieces fit together



A frontend only talks to FastAPI. Everything below the API is internal.

The two things a frontend must do

1. **Log in once** – get a session cookie.
2. **Submit jobs, then poll** – never block on a long request.

Everything else is shape details (payload & result).

Auth model

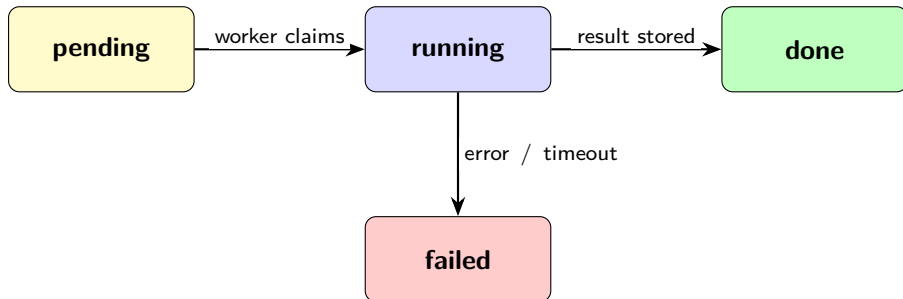
- ▶ Register → Log in → cookie `auth_token` (HttpOnly, 6 h).
- ▶ Browser auto-sends it. SPAs use `credentials: 'include'`.
- ▶ Non-browser clients: `Authorization: Bearer <access_token>`.
- ▶ Any 401 → prompt re-login.

Auth — endpoints

POST	/auth/register	{username, password}	-> {id, username}
POST	/auth/login	{username, password}	->
		{access_token, ...} + cookie	
GET	/auth/me		-> {id, username}
POST	/auth/logout		-> clears cookie

Rules: username ≥ 3 , password ≥ 6 . 409 if taken. 401 if wrong.

Job lifecycle



- ▶ One job runs at a time. Others wait with a `queue_position`.
- ▶ Per-job timeout: 1 hour. Stuck jobs are cleared automatically.
- ▶ Jobs are saved in the database — a page refresh loses nothing.

Endpoints (16 total)

auth

POST /auth/register | **POST** /auth/login | **GET** /auth/me | **POST** /auth/logout

jobs (the core)

POST /api/jobs	submit a simulation
GET /api/jobs	your job history
GET /api/jobs/{id}	one job (with result)
GET /api/jobs/{id}/progress	live incidents processed
GET /api/jobs/queue/status	queue snapshot

inputs / lookups

POST /api/incidents/get-incidents	historical (CSV)
POST /api/incidents/generate-incidents	synthetic (CSV)
POST /api/incidents/process-incidents	stats for a CSV
GET /api/stations/get-stations	list station CSVs
GET /api/stations/get-shapes	list geojson overlays

system

GET /health **GET** /version

Submit a job — run-simulation

```
POST /api/jobs
{
  "kind": "run-simulation",
  "payload": {
    "models":      { "incident": "historical_incidents",
                     "travelTime": "OSRM",
                     "serviceTime": "ml_based" },
    "date_range":  { "start_date": "2024-06-01",
                     "end_date":   "2024-06-03" },
    "incident_type": "ems_fire",           // or "fire"
    "dispatch_policy": "nearest",          // or "firebeats"
    "station_data":   "default_stations", // or "custom_stations"
    "disable_ems":    false
  },
  "priority": 0
}
```

Returns {id, status:"pending", queue_position, ...} instantly.

Submit a job — run-comparison

```
POST /api/jobs
{
  "kind": "run-comparison",
  "payload": {
    "baseline": { /* SimConfig – usually default_stations */ },
    "newConfig": { /* SimConfig – your change to evaluate */ }
  }
}
```

Both legs run in parallel. Result has `baseline`, `newConfig`, and a precomputed `comparison.overall_metrics` with Δ + `improved` booleans.

Custom stations

- ▶ Set `station_data`: "custom_stations".
- ▶ Provide `stations`: [...] — each {id, name, lat, lon, apparatus}.
- ▶ Apparatus types: Engine, Truck, Rescue, Hazard, Squad, FAST, Medic, Brush, Boat, UTV, REACH, Chief.
- ▶ **Rule**: fire incidents need an Engine; EMS need a Medic. Otherwise the sim won't terminate (will time out and fail).

Polling: status + live progress

```
GET /api/jobs/42
```

```
-> { id, status, payload, result, error,  
      duration_seconds, queue_position, ... }
```

```
GET /api/jobs/42/progress
```

```
-> { status, processed, total, percent,  
      legs: { simulation: {processed, total} } }
```

- ▶ Poll every ~3s while active; 15s when idle.
- ▶ percent reaches 100% before resolution finishes. Trust status, not the bar.

Result shape — run-simulation

```
{
  "status": "success",
  "total_incidents": 810,
  "average_response_time": 264.06,
  "coverage_percent": 72.84,
  "P90_continuous": 446.2,
  "duration_seconds": 38.16,

  "station_report": [ { "Station 01": { travel_time_mean,
                                     incident_count, travel_times[],
                                     incident_locations[] } }, ... ],

  "vehicle_report": [ /* per-apparatus metrics */ ],
  "average_response_time_per_incident_type": [ ... ]
}
```

Result shape — comparison summary

```
"comparison": {  
  "overall_metrics": {  
    "average_response_time": { baseline, new, difference,  
                               percent_change, improved },  
    "coverage_percent":      { ... },  
    "p90_response_time":    { ... }  
  },  
  "station_comparison":      [ ... per-station deltas ... ],  
  "incident_type_comparison": [ ... ],  
  "improvements": { response_time_improved,  
                    stations_with_better_response, ... },  
  "summary": { overall_assessment: "improved|degraded|mixed",  
                key_findings: [...] }  
}
```

cURL — end to end

```
B=http://localhost:8000
```

```
curl -X POST $B/auth/register -H 'Content-Type:
  application/json' \
  -d '{"username":"demo","password":"demo-pw1"}'
```

```
curl -c jar -X POST $B/auth/login -H 'Content-Type:
  application/json' \
  -d '{"username":"demo","password":"demo-pw1"}'
```

```
JID=$(curl -b jar -X POST $B/api/jobs -H 'Content-Type:
  application/json' \
  -d @sim.json | jq -r .id)
```

```
curl -b jar $B/api/jobs/$JID/progress
```

```
curl -b jar $B/api/jobs/$JID      # poll until status=done
```

Errors to design for

Code	When	UI
401	missing / expired session	redirect to login
404	job not yours / not found	404 view
409	username taken	inline field error
422	request validation	highlight bad field
job failed	runaway / bad config / timeout	show error text

A failed job is HTTP 200 with `status:"failed"` and a human error.

Run the backend — prereqs

- ▶ Python 3.10+.
- ▶ C++ simulator binary at `data/fire_simulator` (executable).
- ▶ Data assets in `data/` (incidents, stations, ONNX, geojson).
- ▶ OSRM reachable (bundled compose runs it on host 8080).
- ▶ Node 18+ if you'll run the example UI.

Run it — local

```
git clone -b backend-v2 https://github.com/jptalusan/fire_demo3_api.git
cd fire_demo3_api

cp .env.example .env          # edit SECRET_KEY for non-dev
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".*[dev]"

set -a && source .env && set +a    # load BACKEND_PORT etc. into shell

# terminal 1
uvicorn backend.main:app --host 0.0.0.0 --port $BACKEND_PORT

# terminal 2
python -m worker.main
```

Port from `.env`; change `BACKEND_PORT` to use a different one. `storage/`, `logs/`, `data/` auto-created on first start.

Run it — Docker

```
cp .env.example .env          # set BACKEND_PORT / OSRM_HOST_PORT  
    here  
docker compose up --build
```

Brings up backend (\$BACKEND_PORT, default 8000), worker, and osrm (\$OSRM_HOST_PORT, default 8080). Host directories mounted: ./data, ./storage, ./logs. Put data files in ./data before up.

Verify it's alive

```
curl http://localhost:$BACKEND_PORT/health      #  
  {"status":"ok"}  
open http://localhost:$BACKEND_PORT/docs        # interactive  
  Swagger  
curl http://localhost:$BACKEND_PORT/api/v1/openapi.json | head
```

/docs = best place to explore live. /api/v1/openapi.json = feed to client generators (openapi-typescript).

Frontend dev against this API

```
# in your own frontend repo  
fetch('http://localhost:8000/api/jobs', {  
  method: 'POST',  
  credentials: 'include',          // cookie auth  
  headers: {'Content-Type': 'application/json'},  
  body: JSON.stringify({ kind, payload })  
})
```

- ▶ Same-site cookie: page host & API host must match (use 127.0.0.1 on both, or both localhost).
- ▶ Generate a typed client:
npx openapi-typescript openapi.json -o api.ts

Where to look for details

- ▶ `/docs` – live Swagger UI with request examples.
- ▶ `API_REFERENCE.md` – every field, enum, default, full result schemas.
- ▶ `README.md` – running the backend, troubleshooting.
- ▶ `/api/v1/openapi.json` – spec for client generation.

Common gotchas

Symptom	Fix
Job stuck pending	worker not running
No such file or directory	data file missing in data/
Connection refused to OSRM	wrong OSRM_HOST/OSRM_PORT
Orphaned: no live worker	old worker crashed; resubmit
Job times out after 1 h	config can't resolve (e.g. no Engine for fire)
401 on every call	session expired; log in again

Summary for a frontend developer

- ▶ Endpoints + shapes: `/docs` (live) or `API_REFERENCE.md` (text).
- ▶ Authentication: log in once; send the cookie on every request.
- ▶ Any long task: POST to `/api/jobs`, then poll `/api/jobs/{id}` until `status="done"`.
- ▶ Generate a client from the spec at `/api/v1/openapi.json`.

Questions? Open `/docs` and try the endpoint.